TP3 — Le DICP mis à l'épreuve : ce que Git protège, et ce qu'il ne protège pas

Sommaire

Objectifs . . . 1
Prérequis . . . 2
Contexte . . . 2
Étape 0 — Le dépôt de travail . . . 2
Étape 1 — I comme Intégrité : l'empreinte . . . 3
1.1 Une empreinte pour chaque contenu . . . 3
1.2 La chaîne : chaque commit tient le précédent . . . 4
1.3 Bonus — saboter un objet et se faire prendre . . . 4
1.4 Réparer le dépôt avant de continuer . . . 5
Étape 2 — D comme Disponibilité : la copie complète . . . 6
Étape 3 — C comme Confidentialité : le trou dans la raquette . . . 6
Étape 4 — P comme Preuve : traçable, mais falsifiable . . . 7
4.1 Ce que Git trace . . . 7
4.2 Ce que Git ne garantit pas . . . 7
Étape 5 — La grille et le rendu . . . 8
Livrable . . . 9
Mémo . . . 9

Le TD4 vous a fait **nommer** les critères DICP menacés dans douze scénarios. Ce TP vous fait **tester** les quatre critères sur un outil que vous utilisez tous les jours : **Git**. Vous allez découvrir que Git est excellent sur deux critères, correct sur un troisième — et **totalement absent** sur le quatrième.

Objectifs

🎯 À la fin de ce TP, vous saurez :

- montrer **par l'expérience** comment Git garantit l'**intégrité** (empreintes chaînées) et la **disponibilité** (copies complètes) ;
- montrer que Git n'apporte **aucune confidentialité** ;
- mesurer la **limite de la preuve** : fabriquer un commit signé du nom de quelqu'un d'autre ;
- remplir une **grille DICP** argumentée sur un outil réel.

⏳ Durée : **1 heure**.

Prérequis

- **TP0** fait ; **Cours 4** et **TD4 — Identifier la problématique DICP** faits.
- **Git Bash** ouvert.

Contexte

Le **DPO** de MyCallCenter veut inscrire au registre la phrase suivante :

> « Mesure de sécurité : les données sont versionnées dans Git. »

Avant de la valider, il vous demande de **vérifier ce que cette phrase garantit réellement**, critère DICP par critère DICP. Une mesure de sécurité que l'on n'a pas testée n'est pas une mesure de sécurité.

Étape 0 — Le dépôt de travail

⏳ ~5 min

```
# Se placer sur le Bureau (et non dans le home réseau P:)
cd "$(cygpath -u "$USERPROFILE")/Desktop"
pwd        # doit afficher /c/Users/<votre_nom>/Desktop

# Créer le dossier du bloc 3 et s'y placer
mkdir -p bloc3-sio1
cd bloc3-sio1
mkdir tp3-dicp
cd tp3-dicp
git init
git config user.name "Lea Moreau"
git config user.email "l.moreau@mycallcenter.fr"

echo "Virement de 1000 euros vers FR76 3000 1007 9412 3456 7890 185" > virement.txt
git add virement.txt
git commit -m "Enregistre l'ordre de virement fournisseur"

echo 'DB_PASSWORD=Etude2026!prod' > config.env
git add config.env
git commit -m "Ajoute la configuration de la base"
```

Vous avez un dépôt de **2 commits**. C'est votre banc d'essai.

⚠ *Attention aux guillemets sur cette ligne*

Le mot de passe contient un `!`. Dans Git Bash, `!` déclenche le **rappel d'historique** : entre guillemets **doubles**, `!prod` est compris comme « rejoue la commande qui commençait par `prod` », et Bash répond :

```
bash: !prod: event not found
```

La commande échoue, le fichier n'est pas créé, et le `git add` qui suit se plaint d'un fichier introuvable. D'où les guillemets **simples** ici : entre apostrophes, Bash ne touche à rien.

Règle à retenir : **guillemets simples pour un texte contenant `!`, `$` ou `` ` ``.**

Étape 1 — I comme Intégrité : l'empreinte

⏳ ~15 min

1.1 Une empreinte pour chaque contenu

Git calcule pour chaque contenu une **empreinte** : une suite de 40 caractères hexadécimaux, obtenue par une **fonction de hachage**.

```
git hash-object virement.txt
```

Notez les 8 premiers caractères. Maintenant, changez **un seul chiffre** de l'IBAN (le `185` final devient `186`) et recalculez :

```
echo "Virement de 1000 euros vers FR76 3000 1007 9412 3456 7890 186" > virement.txt
git hash-object virement.txt
```

a. Comparez les deux empreintes. Ressemblent-elles l'une à l'autre ?

b. Une modification **minuscule** du contenu produit-elle une modification **minuscule** de l'empreinte ?



L'effet avalanche

C'est la propriété qui rend le hachage utile : **le moindre changement rend l'empreinte méconnaissable**. Impossible de « modifier discrètement » un contenu en conservant son empreinte.

Vous retrouverez exactement cette propriété — et le mot **avalanche** — dans le module **Intégrité et hachage**, avec `sha256sum` et OpenSSL. Git s'en sert ici sans le dire.

Remettez le bon IBAN avant de continuer :

```
echo "Virement de 1000 euros vers FR76 3000 1007 9412 3456 7890 185" > virement.txt
```

1.2 La chaîne : chaque commit tient le précédent

Regardons ce qu'un commit contient **réellement** :

```
git cat-file -p HEAD
```

```
tree 53e457c53cc2bc04388fb31bbb2f2516c5c3c89a
parent 74bc541f6206f865c0992b71f6fe3f353c8a6e69
author Lea Moreau <l.moreau@mycallcenter.fr> 1788460119 +0200
committer Lea Moreau <l.moreau@mycallcenter.fr> 1788460119 +0200

Ajoute la configuration de la base
```

Comparez maintenant la ligne `parent` avec :

```
git log --oneline
```

a. Que reconnaissez-vous dans la ligne `parent` ?

b. **Raisonnement à faire.** Un attaquant veut modifier le **premier** commit sans que personne ne le voie. En changeant son contenu, il change son empreinte. Or cette empreinte est **écrite à l'intérieur** du commit suivant… Expliquez, en deux phrases, pourquoi il ne peut pas s'arrêter là.



C'est une **chaîne de blocs** : chaque maillon contient l'empreinte du précédent. Modifier un maillon oblige à refabriquer **tous les suivants**. C'est le principe qui sécurise aussi les registres de type *blockchain*.

Verdict pour le critère I : Git protège très bien l'intégrité.

1.3 Bonus — saboter un objet et se faire prendre

Si vous avez de l'avance :

```
git fsck && echo "Depot intact"

OID=$(git rev-parse HEAD~1:virement.txt)
CHEMIN=".git/objects/${OID:0:2}/${OID:2}"
chmod +w "$CHEMIN"
printf 'X' >> "$CHEMIN"
```

```
git fsck
```

Git détecte l'altération : `object corrupt or missing`. Il ne peut pas **empêcher** la modification d'un fichier sur le disque, mais il la **détecte** — c'est déjà énorme.



Cette manipulation **casse volontairement** votre dépôt d'essai. Ne la faites que dans `tp3-dicp`, jamais dans un dépôt qui contient du vrai travail.

Le dépôt reste cassé tant que vous ne l'avez pas réparé : l'étape 2 ne fonctionnera pas (le clone se crée, mais le checkout échoue et la corruption est recopiée). Réparez avant de continuer.

1.4 Réparer le dépôt avant de continuer

L'objet détruit contenait le texte de `virement.txt`, et ce fichier est toujours sur votre disque. On peut donc **reconstruire l'objet à l'identique** :

```
OID=$(git rev-parse HEAD~1:virement.txt)
CHEMIN=".git/objects/${OID:0:2}/${OID:2}"

chmod +w "$CHEMIN"
rm "$CHEMIN"                  # on jette l'objet abîmé

git hash-object -w virement.txt   # Git le recalcule depuis le fichier
git fsck && echo "Depot repare"
```

`git hash-object -w` renvoie **exactement la même empreinte** qu'avant le sabotage : c'est la preuve que le contenu est identique. Si l'empreinte affichée diffère de `$OID`, c'est que votre `virement.txt` n'est plus celui d'origine — remettez le bon IBAN (`···7890 185`) et relancez la commande.



Pourquoi ça marche, et ce que ça ne prouve pas

On n'a rien « réparé » au sens de la médecine : on a **refabriqué** l'objet à partir d'une **autre copie** du contenu, celle du dossier de travail. Sans cette copie, l'objet était perdu.

C'est exactement la leçon de l'étape 2 : l'intégrité permet de **détecter** le problème, seule la **redondance** permet d'en sortir. Si vous aviez saboté un objet dont aucun fichier ne portait plus le contenu, il aurait fallu le récupérer depuis… une sauvegarde.

En dernier recours, si la réparation échoue, reconstruisez le dépôt d'essai : il ne contient que deux commits. Remontez d'un cran (`cd ..`), supprimez le dossier (`rm -rf tp3-dicp`) et rejouez l'**étape 0**. Trois minutes.

Étape 2 — D comme Disponibilité : la copie complète

⏳ ~10 min

```
cd ..                       # on remonte dans bloc3-sio1
git clone tp3-dicp tp3-sauvegarde
cd tp3-sauvegarde
git log --oneline
```

a. La sauvegarde contient-elle seulement le dernier état, ou **tout l'historique** ?

Simulons la panne : le poste de Léa est chiffré par un rançongiciel. On supprime son dépôt de travail.

```
cd ..                       # on remonte dans bloc3-sio1
rm -rf tp3-dicp
ls
```

Puis on repart de la copie :

```
git clone tp3-sauvegarde tp3-dicp
cd tp3-dicp
git log --oneline
cat virement.txt
```

a. Qu'avez-vous perdu ?

b. En quoi un dépôt Git **cloné par cinq développeurs** est-il différent d'un fichier Word posé sur un serveur unique, du point de vue de la **disponibilité** ?



Verdict pour le critère D : Git aide beaucoup, parce que chaque clone est une **copie complète et autonome**. Attention toutefois : ce n'est pas une politique de sauvegarde à soi seul. Un clone ne protège pas d'une suppression **volontairement propagée** (voir TP5), et il ne sauvegarde que ce qui a été **commité**.

Étape 3 — C comme Confidentialité : le trou dans la raquette

⏳ ~10 min

Vous êtes maintenant l'attaquant. Vous avez récupéré une clé USB contenant le dossier `tp3-sauvegarde` — sans mot de passe, sans compte, sans accès au serveur.

```
cd ../tp3-sauvegarde       # depuis tp3-dicp
git log --oneline
cat config.env
git show HEAD:config.env
```

a. Vous a-t-on demandé le moindre mot de passe pour lire ce contenu ?

b. Le fichier de configuration contient un mot de passe de production. Sous quelle forme est-il stocké dans le dépôt ?

c. Reprenez la conclusion du **TP1** : si ce mot de passe avait été « supprimé » par un commit ultérieur, seriez-vous quand même capable de le lire ? Avec quelle commande ?



Verdict pour le critère C

Git n'apporte aucune confidentialité. Absolument rien n'est chiffré dans un dépôt : ni les fichiers, ni l'historique, ni les messages de commit.

Ce qui est chiffré, c'est le **transport** (HTTPS ou SSH), pendant les quelques secondes du `push`. Une fois arrivé, tout est **en clair** — sur la forge, sur chaque poste, dans chaque clone, sur chaque sauvegarde.

La confidentialité doit donc venir d'**ailleurs** : contrôle d'accès (dépôt privé, habilitations) et **chiffrement** des données elles-mêmes — c'est l'objet du module **Chiffrement symétrique (AES)**.

Étape 4 — P comme Preuve : traçable, mais falsifiable

⏳ ~15 min

4.1 Ce que Git trace

```
cd ../tp3-dicp             # depuis tp3-sauvegarde
git log --pretty=format:'%h | %an <%ae> | %ad | %s' --date=iso
```

Chaque commit porte un **auteur**, un **e-mail**, une **date**, un **message**. C'est le pilier **P** du Cours 4 : on sait qui a fait quoi, et quand.

4.2 Ce que Git ne garantit pas

Vous êtes un salarié qui veut faire passer une nouvelle procédure en la faisant attribuer à sa direction.

```
echo "Nouvelle regle : virements valides sans double controle" > procedure.txt
```

```
git add procedure.txt
git commit -m "Valide la nouvelle procedure de virement" \
  --author="Directeur Financier <df@mycallcenter.fr>"

git log -1
```

a. Quel nom apparaît à la ligne `Author` ?

b. Ce commit a-t-il exigé le moindre mot de passe, la moindre autorisation du directeur financier ?

Cherchons la trace du vrai coupable :

```
git log -1 --pretty=format:'auteur=%an | committer=%cn'
```

a. Git distingue l'**auteur** et le **validateur** (*committer*). Lequel des deux correspond à la personne réellement assise devant le poste ?

b. `git log` **par défaut** affiche-t-il le validateur ? Un auditeur pressé verrait-il la supercherie ?



Traçabilité n'est pas non-répudiation

Le nom d'un commit vient de `git config user.name` : c'est une **chaîne de caractères déclarée par l'utilisateur**, jamais vérifiée. Le champ `committer` peut lui aussi être forgé.

Donc, précisément :

- Git assure la **traçabilité** : la preuve est solide contre l'**erreur** et l'**oubli** (« qui a introduit ce bug ? », « quand cette ligne a-t-elle été ajoutée ? ») ;
- Git n'assure **pas** la **non-répudiation** : la preuve est faible contre la **malveillance**. On peut nier (« ce n'est pas moi »), et on peut accuser (« regardez, c'est signé de son nom »).

Verdict pour le critère P : partiellement assuré. Pour aller jusqu'à la non-répudiation, il faut **signer** les commits (`git commit -S`) avec une clé **privée** que l'auteur seul détient — c'est le module **RSA : signature**.

Étape 5 — La grille et le rendu

⏳ ~5 min

Complétez la grille suivante et déposez-la dans votre dépôt `bloc3-cyber`, dossier `tp3-dicp/`, sous le nom `grille-dicp-git.md`.

Critère	Git le garantit ?	Preuve tirée du TP (commande + observation)	Ce qu'il faut ajouter
Disponibilité			

Critère	Git le garantit ?	Preuve tirée du TP (commande + observation)	Ce qu'il faut ajouter
I ntégrité			
C onfidentialité			
P reuve			

Pour la colonne « Git le garantit ? », utilisez : **Oui** / **Partiellement** / **Non**.

Ajoutez, sous la grille, une **conclusion de 5 lignes** répondant au DPO : la phrase « les données sont versionnées dans Git » est-elle une mesure de sécurité suffisante pour le registre ? Que faut-il écrire à la place ?

```
cd ../bloc3-cyber          # le dépôt de rendu, créé au TP1
mkdir -p tp3-dicp
# ... déposez grille-dicp-git.md dans ce dossier ...
git add tp3-dicp
git commit -m "TP4 : grille DICP appliquee a Git"
git push
```

Livrable

📦 À rendre dans `bloc3-cyber/tp3-dicp/` :

- ☐ `grille-dicp-git.md` : les 4 critères, chacun avec une **commande** et une **observation** issues du TP.
- ☐ La conclusion au DPO (5 lignes).

Table 1. Auto-évaluation avant de rendre

Vérification	OK ?
Chaque ligne de ma grille cite une commande que j'ai réellement lancée	[ ]
J'ai expliqué l'effet avalanche avec mes deux empreintes	[ ]
J'ai expliqué pourquoi la ligne `parent` protège l'historique	[ ]
J'ai bien mis **Non** en confidentialité, et je sais dire pourquoi	[ ]
J'ai distingué **traçabilité** et **non-répudiation**	[ ]

Mémo

Critère	Git	Commande de démonstration
D	Oui (clones complets)	`git clone`, puis restauration après suppression
I	Oui (empreintes chaînées)	`git hash-object`, `git cat-file -p HEAD`, `git fsck`
C	**Non** (tout en clair)	`git show HEAD:config.env`
P	Partiellement (falsifiable)	`git log`, `git commit --author="…"`

Ce qui manque	Où on le verra
Chiffrement des contenus	Module **Chiffrement symétrique (AES)**
Vérification de l'intégrité hors Git	Module **Intégrité et hachage**
Non-répudiation de l'auteur	Module **RSA : signature et hybride**

BTS SIO - ENC Paris — Bloc 3 - Cybersécurité des services informatiques — 2026-2027